

## # Sobre o projeto

### ## Resumo executivo

O Minhocao Wiki é uma plataforma de perguntas e respostas com RAG (Retrieval-Augmented Generation) focada em conteúdos da UnB.

A solução combina backend FastAPI, frontend Next.js e banco PostgreSQL, com pipeline de ingestão de PDFs para indexação vetorial em Pinecone e geração de respostas com LLM.

Objetivo principal:

- Permitir que usuários façam perguntas em linguagem natural e recebam respostas contextualizadas a partir de documentos oficiais/acadêmicos.

### ## Autor (destaque principal)

Edilberto Almeida Cantuária é o autor principal e referência central deste projeto.

Contribuição do autor no projeto:

- Definição da proposta: criar uma wiki conversacional com base documental.
- Direcionamento da arquitetura full stack (API + Web + banco + infraestrutura Docker).
- Organização do fluxo de autenticação (JWT e Google) e histórico de conversas.
- Construção e evolução do pipeline de ingestão de documentos para o RAG.
- Estruturação da base para escalabilidade, manutenção e colaboração futura.

Impacto da autoria:

- O projeto possui identidade técnica clara.
- Há foco em utilidade prática para consulta acadêmica.
- Existe preocupação com rastreabilidade (logs/manifests) e operação local simples.

Credito de autoria para registro institucional e documental:

- Autor: Edilberto Almeida Cantuaria

## **## Problema que o projeto resolve**

Antes da plataforma, a busca por informacoes em documentos extensos pode ser lenta e fragmentada.

Com o Minhocao Wiki, a consulta passa a ser:

- Conversacional
- Contextual
- Persistente (com historico por usuario)
- Integrada a um conjunto de fontes indexadas

## **## Escopo funcional atual**

Funcionalidades principais:

- Cadastro e login por email/senha.
- Login com Google.
- Emissao e validacao de JWT.
- Criacao, listagem, leitura e remocao de conversas.
- Chat com streaming de resposta.
- Persistencia de mensagens (usuario e assistente).
- Ingestao de PDFs para alimentar o indice vetorial.

## **## Arquitetura geral**

Camadas:

- Web (Next.js 16): interface de autenticacao e chat.
- API (FastAPI): regras de negocio, auth, conversas e chat RAG.
- Banco (PostgreSQL 16): usuarios, conversas e mensagens.
- Vetor/LLM (Pinecone + OpenAI): recuperacao semantica e geracao.

- Integracao externa: Google tokeninfo para login social.

Fluxo simplificado:

1. Usuario autentica no frontend.
2. Frontend cria/seleciona conversa.
3. Pergunta e enviada ao endpoint de chat.
4. API consulta contexto e fontes relevantes no RAG.
5. Resposta e transmitida em streaming.
6. Pergunta e resposta ficam salvas no banco.

## **## Estrutura de pastas**

Raiz:

- api/: backend, docs, ingestao e testes.
- web/: frontend e componentes UI.
- docker-compose.yml: orquestracao local.
- servers.json: suporte ao pgAdmin.

Backend (api/):

- app/main.py: inicializacao FastAPI.
- app/routers/: rotas de auth, chat e conversas.
- app/services/: regras de negocio.
- app/models.py: modelos do banco.
- app/config.py: leitura de configuracoes.
- docs/api-doc.md: documentacao detalhada de endpoints.
- ingestion/ingest\_documents.py: pipeline de ingestao RAG.
- tests/: testes de API e fluxo de chat.

Frontend (web/):

- app/: paginas principais (login, registro e chat).
- app/api/: rotas proxy para backend.
- components/: interface de chat e componentes reutilizaveis.
- contexts/auth-context.tsx: estado de autenticacao.
- hooks/ e lib/: utilitarios de suporte.

## **## Stack tecnologica**

### Backend:

- Python 3.11
- FastAPI
- SQLAlchemy
- Uvicorn
- LangChain
- Pinecone client
- OpenAI SDK

### Frontend:

- Next.js 16
- React 19
- TypeScript
- Tailwind CSS
- shadcn/ui + Radix UI

### Infra:

- Docker Compose
- PostgreSQL 16
- pgAdmin 4

## **## Infraestrutura local com Docker**

Servicos definidos:

- db: PostgreSQL com volume persistente.
- pgadmin: administracao visual do banco.
- api: backend FastAPI com montagens para config/docs/logs.
- web: frontend Next.js com API interna apontando para o servico api.

Portas padrao:

- Web: 3000
- API: 8000
- PostgreSQL: 5432
- pgAdmin: 5050

## **## Configuracao e segredos**

Variaveis criticas:

- DATABASE\_URL
- JWT\_SECRET\_KEY
- GOOGLE\_CLIENT\_ID
- OPENAI\_API\_KEY
- PINECONE\_API\_KEY
- INDEX\_NAME

Fontes de configuracao:

- Arquivo de ambiente na raiz (.env).
- Arquivo api/config.yaml (com exemplo em config.yaml.exemple).
- Variaveis de ambiente podem sobrescrever valores de arquivo.

## **## Pipeline de ingestao (RAG)**

Script principal:

- api/ingestion/ingest\_documents.py

Comportamento do pipeline:

- Descobre PDFs em api/docs/.
- Calcula hash de arquivo para detectar alteracoes.
- Compara com document\_manifest.json.
- Identifica documentos novos, alterados, removidos e inalterados.
- Remove do indice vetorial o que foi removido/atualizado.
- Carrega PDFs com PyPDFLoader.
- Divide em chunks (chunk\_size=1000, overlap=150).
- Gera embeddings via OpenAI.
- Faz upsert em lotes no Pinecone.
- Atualiza manifesto de documentos.
- Gera log de auditoria em api/logs/.

Ganhos do desenho adotado:

- Evita reprocessamento desnecessario.
- Permite manutencao incremental da base vetorial.
- Melhora rastreabilidade operacional.

## **## API e contratos**

Padrao:

- API REST com JSON para cadastro/login/conversas.
- Streaming de texto no endpoint de chat.

Rotas centrais:

- POST /auth/register

- POST /auth/login
- POST /auth/token
- POST /auth/google
- POST /conversations
- GET /conversations
- GET /conversations/{conversation\_id}
- DELETE /conversations/{conversation\_id}
- POST /chat (streaming)

Documentacao operacional:

- Swagger local em /docs.
- Detalhamento completo em api/docs/api-doc.md.

## **## Frontend e experiencia de uso**

Capacidades de interface:

- Login e cadastro.
- Botao de login Google.
- Sidebar com historico de conversas.
- Renderizacao incremental de resposta.
- Estrutura de componentes reutilizaveis para consistencia visual.

Integracao web-api:

- Rotas internas no Next atuam como proxy para API.
- Requisicoes autenticadas usam Bearer token.

## **## Banco de dados e persistencia**

Camadas de dados:

- Usuarios

- Conversas
- Mensagens

Resultado pratico:

- Cada interacao relevante fica registrada.
- O usuario pode retomar contexto historico.

## **## Qualidade, logs e operacao**

Observabilidade atual:

- Logs de ingestao por execucao com timestamp.
- Arquivo de logs RAG em api/logs/rag\_logs.jsonl.
- Manifesto de documentos para controle incremental.

Testes:

- Testes backend em api/tests.
- Casos para API geral e fluxo de chat.

## **## Seguranca e autenticacao**

Controles implementados:

- JWT para rotas protegidas.
- Hash de senha com passlib/bcrypt.
- Login social com validacao de token Google (audiencia e email verificado).

Boas praticas recomendadas para continuidade:

- Nao versionar segredos.
- Rotacionar chaves periodicamente.
- Separar configuracao por ambiente (dev/hml/prod).



## **## Como executar (resumo pratico)**

Caminho rapido com Docker:

1. Preparar variaveis em .env.
2. Ajustar api/config.yaml.
3. Subir com docker compose up -d --build.
4. Abrir web em localhost:3000 e Swagger em localhost:8000/docs.

Ingestao manual:

1. Entrar em api/.
2. Rodar python ingestion/ingest\_documents.py.

## **## Estado atual e maturidade**

Maturidade tecnica:

- Projeto funcional end-to-end (auth + conversa + RAG + persistencia).
- Arquitetura modular com separacao de responsabilidades.
- Pronto para evolucao incremental.

Possiveis proximos passos:

- Observabilidade centralizada (metrics e tracing).
- Testes e2e do frontend.
- Politicas de rate limit e hardening de seguranca.
- Pipeline CI/CD com validacao automatizada.

## **## Registro final de autoria**

Este projeto representa uma implementacao completa de assistente conversacional com base documental, e sua identidade tecnica e estrategica esta diretamente associada ao autor Edilberto Almeida Cantuaria.

Se este documento for utilizado em apresentacoes, portfolio, relatorios academicos ou contextos institucionais, manter o credito explicito ao autor e parte essencial da historia e da integridade do projeto.